

# ADP470S

## Applications Development Practice 4

### COMPREHENSIVE TERM 1 STUDY NOTES

Cape Peninsula University of Technology | 2026

*Data Structures & Algorithms | Java Implementation*

#### TOPICS COVERED IN THIS DOCUMENT:

1. What is an Algorithm?
2. Big O Notation
3. Arrays (1D, 2D, Multi)
4. Recursion vs Iteration
5. Factorial & Fibonacci
6. Linear & Binary Search
7. Bubble Sort
8. Insertion Sort
9. Quick Sort
10. Heap Sort
11. Linked Lists (Singly, Doubly, Circular)

All code snippets include LINE-BY-LINE variable explanations.

Green lines = comments explaining WHY each variable/loop exists.

# 1. What is an Algorithm?

---

An algorithm is a finite, ordered set of clear, unambiguous instructions that solves a specific problem. Think of it as a recipe — it takes inputs, follows steps, and produces an output.

## Five Properties of an Algorithm (must know for definitions!)

| Property      | Meaning                                | Example                     |
|---------------|----------------------------------------|-----------------------------|
| INPUT         | Zero or more well-defined inputs       | $n = 5$ passed to factorial |
| OUTPUT        | At least one result produced           | Returns 120                 |
| DEFINITENESS  | Every step is clear and unambiguous    | No vague instructions       |
| FINITENESS    | Must terminate in finite steps         | Loop must end               |
| EFFECTIVENESS | Each step must be basic and executable | No impossible operations    |

## Why Analyse Algorithms?

- Different algorithms can solve the same problem in different ways
- We want to know: How FAST is it? (Time) | How much MEMORY? (Space)
- Big O Notation is the tool we use to measure and compare this

## 2. Big O Notation — Algorithm Complexity

Big O notation describes how an algorithm's performance **SCALES** as input size  $n$  grows. It ignores constants and focuses on the dominant growth pattern.

### 2.1 Time Complexity vs Space Complexity

| Type             | Measures                          | What to count                    |
|------------------|-----------------------------------|----------------------------------|
| TIME Complexity  | How many operations are performed | Loops, comparisons, method calls |
| SPACE Complexity | How much memory is used           | Variables, arrays, stack frames  |

### 2.2 Common Big O Classes

| Big O         | Name         | Example                          | For $n=1000$         |
|---------------|--------------|----------------------------------|----------------------|
| $O(1)$        | Constant     | Array lookup <code>arr[5]</code> | 1 operation          |
| $O(\log n)$   | Logarithmic  | Binary Search                    | ~10 operations       |
| $O(n)$        | Linear       | Single loop                      | 1,000 operations     |
| $O(n \log n)$ | Linearithmic | QuickSort, HeapSort              | ~10,000 operations   |
| $O(n^2)$      | Quadratic    | Nested loops                     | 1,000,000 operations |
| $O(2^n)$      | Exponential  | Recursive Fibonacci              | Enormous             |

### 2.3 The Golden Rules of Big O

- **Drop constants:**  $O(2n) \rightarrow O(n)$  |  $O(5n^2) \rightarrow O(n^2)$
- **Drop lower-order terms:**  $O(n^2 + n) \rightarrow O(n^2)$
- **Sequential loops ADD:**  $O(n) + O(n) = O(2n) \rightarrow O(n)$
- **Nested loops MULTIPLY:**  $O(n) \times O(n) = O(n^2)$
- **Tripling/halving loop:**  $j = j*3$  or  $j = j/2 \rightarrow O(\log n)$

### 2.4 Big O Code Examples (Exam Favourite — Know All 3!)

#### Example A — Two Sequential Loops $\rightarrow O(n)$

```
// arr = the array we are working with (its length = n)
// sum = a running total we build up - just ONE variable, not an array
// i    = the loop counter; it goes from 0 up to n-1 (every slot once)
// j    = a second independent counter; it also goes 0 to n-1
//      We use i in loop 1 and j in loop 2 - they do the SAME work
```

```
//      but are separate passes, not nested inside each other

int sum = 0;
for (int i = 0; i < n; i++) {    // Loop 1 runs n times → O(n)
    sum = sum + a[i];           // add element at position i to sum
}
for (int j = 0; j < n; j++) {    // Loop 2 also runs n times → O(n)
    sum = sum + a[j];           // another n additions
}
// TOTAL: O(n) + O(n) = O(2n) → drop the constant 2 → ANSWER: O(n)
// KEY RULE: Sequential (one after the other) loops ADD their costs.
```

## Example B — Nested Loops where inner loop triples → $O(n \log n)$

```
// prod = running product (like sum but for multiplication)
// i     = outer loop counter; goes 1, 2, 3, ..., n-1
//       runs n times total → contributes O(n)
// j     = inner loop counter; STARTS at i (not 0!) and TRIPLES each step
//       j goes: i, i*3, i*9, i*27, ... until j >= n
//       Because j multiplies by 3 each time, it only needs log3(n) steps
//       This is logarithmic → O(log n)
// j = j*3 means: j is NOT counting 1,2,3 – it is growing EXPONENTIALLY
//       so the number of steps shrinks → that is the LOG pattern

int prod = 1;
for (int i = 1; i < n; i++) {    // Outer: n iterations → O(n)
    for (int j = i; j < n; j=j*3) { // Inner: j triples → O(log n)
        prod = prod * j;
    }
}
// TOTAL: O(n) × O(log n) = O(n log n)
// KEY RULE: Nested loops MULTIPLY their costs.
```

## Example C — Outer steps of 3, Inner counts down → $O(n^2)$

```
// i     = outer loop counter; starts at 0 and increases by 3 each time
//       so i goes: 0, 3, 6, 9, ..., n-1
//       That means it runs n/3 times (not n) → but 1/3 is a CONSTANT
//       In Big O we DROP constants, so n/3 is still treated as O(n)
// j     = inner loop counter; starts at n-1 and counts DOWN to 1
//       j goes: n-1, n-2, n-3, ..., 1 → that is n steps → O(n)
// j--   = shorthand for j = j - 1 (decrease j by 1 each iteration)
// j > 0 = loop condition: keep going as long as j is greater than 0
// sum   = accumulates i*j products – just to have some real work to do

int sum = 0;
for (int i = 0; i <= n-1; i += 3) { // Outer: n/3 iterations → O(n)
    for (int j = n-1; j > 0; j--) { // Inner: n iterations → O(n)
        sum = sum + (i * j);
    }
}
// TOTAL: O(n/3) × O(n) = O(n2/3) → drop the 1/3 constant → ANSWER: O(n2)
```

## 3. Arrays

An array is a collection of elements of the SAME TYPE stored in CONTIGUOUS (adjacent) memory locations. Each element is accessed by its INDEX, which starts at 0 in Java.

### 3.1 One-Dimensional (1D) Array

Think of a 1D array as a row of boxes, each labelled with an index number.

Index: [0] [1] [2] [3] [4]  
Values: 10 20 30 40 50

arr[0] = 10 (first box)    arr[4] = 50 (last box)

```
// int[] = declares that this is an array that holds integers
// numbers = the NAME we give the array - you choose this
// new int[5] = creates 5 empty slots (all start at 0)
//           The number 5 is the CAPACITY - fixed once created
int[] numbers = new int[5];

// { } syntax lets us fill the array immediately at creation time
// Java counts from 0, so grades[0]=85, grades[1]=90, grades[4]=60
int[] grades = {85, 90, 78, 92, 60};

// grades[0] = go to the array called 'grades', get the item at slot 0
System.out.println(grades[0]); // prints 85
System.out.println(grades[4]); // prints 60

// --- Looping through every element ---
// i = the current index (position), starts at 0
// i < grades.length = stop BEFORE going past the last slot
//           .length tells us how many slots exist (5 here)
// i++ = shorthand for i = i+1, move to the next slot
// arr[i] = get the element at position i
for (int i = 0; i < grades.length; i++) {
    System.out.println(grades[i]); // prints 85, 90, 78, 92, 60
}

// for-each loop: same result, simpler syntax when you don't need 'i'
// g = a temporary variable that holds each element one at a time
for (int g : grades) {
    System.out.println(g);
}
```

| Operation         | Time Complexity | Why                                        |
|-------------------|-----------------|--------------------------------------------|
| Access arr[i]     | O(1)            | Direct: compute memory address in one step |
| Search (unsorted) | O(n)            | May need to check every element            |

|                    |      |                                            |
|--------------------|------|--------------------------------------------|
| Insert at end      | O(1) | Just place value at the known last index   |
| Insert at middle   | O(n) | Must shift all elements to the right first |
| Delete from middle | O(n) | Must shift all elements to the left after  |

## 3.2 Two-Dimensional (2D) Array

A 2D array is an array of arrays — picture a TABLE with rows and columns.

Think of it like a spreadsheet grid:

|         | Col 0 | Col 1 | Col 2 |
|---------|-------|-------|-------|
| Row 0 → | [ 1,  | 2,    | 3 ]   |
| Row 1 → | [ 4,  | 5,    | 6 ]   |
| Row 2 → | [ 7,  | 8,    | 9 ]   |

matrix[1][2] = Row 1, Column 2 = value 6

```
// int[][] = 2D array of integers (array of arrays)
// matrix = name we give it
// Each { } inside the outer { } is ONE ROW of the grid
int[][] matrix = {
    {1, 2, 3}, // Row 0: matrix[0][0]=1, matrix[0][1]=2, matrix[0][2]=3
    {4, 5, 6}, // Row 1: matrix[1][0]=4, matrix[1][1]=5, matrix[1][2]=6
    {7, 8, 9}  // Row 2: matrix[2][0]=7, matrix[2][1]=8, matrix[2][2]=9
};

// matrix[1][2] = go to row 1, then column 2 → value is 6
System.out.println(matrix[1][2]); // Output: 6

// --- Nested loops to visit every cell ---
// row = current row number (outer loop)
// col = current column number (inner loop)
// matrix.length = number of ROWS (3 here)
// matrix[row].length = number of COLUMNS in that row (3 here)
// We need TWO loops because we have TWO dimensions (row + col)
for (int row = 0; row < matrix.length; row++) {
    for (int col = 0; col < matrix[row].length; col++) {
        System.out.print(matrix[row][col] + " ");
    }
    System.out.println(); // move to a new line after finishing each row
}
```

## 3.3 Why We Need Linked Lists (Limitations of Arrays)

- **FIXED SIZE** — once you declare `int[100]`, you cannot grow or shrink it
- **MEMORY WASTE** — declare `int[100]` but use 10 → 90 slots wasted
- **SLOW INSERTS/DELETES** — inserting at position 2 means shifting every element after it → O(n)

- SOLUTION: Linked Lists are DYNAMIC — each node is created on demand, no wasted slots

## 4. Recursion vs Iteration

Both are ways to repeat work. The key question is: should the method call ITSELF (recursion) or should we use a LOOP (iteration)?

| Aspect          | RECURSION                         | ITERATION                     |
|-----------------|-----------------------------------|-------------------------------|
| How it repeats  | Method calls ITSELF               | Uses a for/while LOOP         |
| Stops when      | BASE CASE returns without calling | Loop condition becomes false  |
| Memory (Space)  | $O(n)$ — $n$ stack frames created | $O(1)$ — only loop variables  |
| Speed (Time)    | $O(n)$ — $n$ method calls         | $O(n)$ — $n$ iterations       |
| Stack Overflow? | YES — risk for very large $n$     | NO — no call stack used       |
| Readability     | Elegant, mirrors maths formula    | More lines but easy to follow |
| When to use?    | Trees, graphs, divide & conquer   | Preferred for simple repeats  |

### 4.1 What is a Stack Frame?

Every time a method is called in Java, the JVM (Java Virtual Machine) creates a STACK FRAME. Each frame is like a box that stores the method's local state:

- The RETURN ADDRESS — where to jump back after this method finishes
- The PARAMETERS — the value of  $n$  that was passed in
- LOCAL VARIABLES — any variables declared inside the method

For factorial(4) — Java creates 4 stack frames stacked on top of each other:

TOP → Frame 4: factorial(1)  $n=1$  → BASE CASE: returns 1  
Frame 3: factorial(2)  $n=2$  → waiting: will compute  $2 \times 1$   
Frame 2: factorial(3)  $n=3$  → waiting: will compute  $3 \times 2$   
BOTTOM → Frame 1: factorial(4)  $n=4$  → waiting: will compute  $4 \times 6$

4 frames =  $O(n)$  SPACE used in memory

Each frame does one multiplication =  $O(1)$  each →  $4 \times O(1) = O(n)$  TIME

## 5. Factorial — Recursive & Iterative

Factorial of  $n$  (written  $n!$ ) =  $n \times (n-1) \times (n-2) \times \dots \times 2 \times 1$

Examples:  $5! = 120$  |  $4! = 24$  |  $0! = 1$  |  $1! = 1$

### 5.1 Recursive Factorial — with variable explanations

```
public class Factorial {  
  
    // n      = the number we want the factorial of (passed in by the caller)  
    // WHY recursive: factorial(n) = n * factorial(n-1)  
    //                the problem naturally shrinks by 1 each call  
    // BASE CASE: when n reaches 0 or 1 we STOP and return 1  
    //                without a base case the method would call itself forever  
    //                and crash with java.lang.StackOverflowError  
  
    public static int factorialRecursive(int n) {  
  
        // BASE CASE - stop recursing here, return 1  
        // 0! = 1 and 1! = 1 are mathematical definitions  
        if (n == 0 || n == 1) {  
            return 1;  
        }  
  
        // RECURSIVE CASE  
        // We call the SAME method but with a SMALLER input (n-1)  
        // This is what shrinks the problem toward the base case  
        // n * factorialRecursive(n-1) will only compute once n-1 returns  
        return n * factorialRecursive(n - 1);  
    }  
  
    public static void main(String[] args) {  
        System.out.println(factorialRecursive(5)); // Output: 120  
        System.out.println(factorialRecursive(0)); // Output: 1  
    }  
}
```

#### Execution Trace — factorial(5)

```
factorial(5) → 5 × factorial(4)  calls itself with n=4  
factorial(4) → 4 × factorial(3)  calls itself with n=3  
factorial(3) → 3 × factorial(2)  calls itself with n=2  
factorial(2) → 2 × factorial(1)  calls itself with n=1  
factorial(1) = 1 ← BASE CASE hit! Returns 1 immediately.  
factorial(2) = 2 × 1 = 2          (frame 4 popped, frame 3 resumes)  
factorial(3) = 3 × 2 = 6          (frame 3 popped, frame 2 resumes)  
factorial(4) = 4 × 6 = 24         (frame 2 popped, frame 1 resumes)  
factorial(5) = 5 × 24 = 120 ← FINAL ANSWER
```

### 5.2 Iterative Factorial — with variable explanations

```

public class Factorial {

    // n      = the number we want factorial for (input)
    // result = this is our ACCUMULATOR – it builds up the answer
    //         we start it at 1 because multiplying by 1 doesn't change anything
    //         and because 0! = 1! = 1 (handled by starting at 1 before loop)
    // i      = the loop counter; represents the current multiplier
    //         WHY start at 2? Because multiplying by 1 adds nothing.
    //         i = 2 means we begin with: result = 1 × 2 = 2
    // i <= n = keep looping WHILE i is less than or equal to n
    //         when i passes n, we have multiplied all the way up to n – done
    // i++    = increase i by 1 each time through the loop
    // result = result * i means: take the running total and multiply by i
    //         this is equivalent to: result *= i

    public static long factorialIterative(int n) {
        long result = 1; // Start at 1 (0! = 1! = 1 by definition)

        // Loop from 2 up to n (inclusive)
        // i=2 first because result already covers 0! and 1! = 1
        for (int i = 2; i <= n; i++) {
            result = result * i; // build up: 1×2, then 2×3, then 6×4 ...
        }

        return result; // e.g. for n=5: 1×2×3×4×5 = 120
    }

    public static void main(String[] args) {
        System.out.println(factorialIterative(5)); // Output: 120
        System.out.println(factorialIterative(0)); // Output: 1
    }
}

```

### Step-by-Step Trace — factorialIterative(5)

| Step | i value | Calculation                | result after step |
|------|---------|----------------------------|-------------------|
| Init | —       | result starts at 1         | result = 1        |
| i=2  | 2       | result = 1 × 2             | result = 2        |
| i=3  | 3       | result = 2 × 3             | result = 6        |
| i=4  | 4       | result = 6 × 4             | result = 24       |
| i=5  | 5       | result = 24 × 5, loop ends | result = 120 ✓    |

## 5.3 Comparison Summary

|                  | Recursive             | Iterative                  |
|------------------|-----------------------|----------------------------|
| Time Complexity  | O(n)                  | O(n)                       |
| Space Complexity | O(n) — n stack frames | O(1) — one result variable |
| Stack Overflow?  | YES for large n       | NO                         |

| Preferred? | Avoid for simple problems | YES — always preferred here |
|------------|---------------------------|-----------------------------|
|------------|---------------------------|-----------------------------|

EXAM ANSWER: ITERATIVE is preferred for factorial because:

1. Space: uses  $O(1)$  vs  $O(n)$  — no risk of `StackOverflowError`
2. Speed: no method-call overhead — just one loop variable
3. Scalability: works for  $n = 1,000,000$  without crashing

Use recursion ONLY when the problem is NATURALLY recursive (trees, graphs).

## 6. Fibonacci Sequence

Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34 ...

Rule: each number = sum of the two before it.  $\text{fib}(n) = \text{fib}(n-1) + \text{fib}(n-2)$

Base cases:  $\text{fib}(0) = 0$  |  $\text{fib}(1) = 1$

### 6.1 Recursive Fibonacci — with variable explanations

```
// n = the position in the sequence we want (e.g. n=5 → answer is 5)
// BASE CASES: fib(0)=0 and fib(1)=1 are defined, not computed
//           without them the method would call itself forever
// TWO recursive calls per method = the tree of calls DOUBLES each level
//   fib(5) calls fib(4) AND fib(3)
//   fib(4) calls fib(3) AND fib(2) ← fib(3) already computed above!
//   This repeated work is why recursive fib is  $O(2^n)$  — extremely slow

public static int fibRecursive(int n) {
    if (n == 0) return 0; // BASE CASE: by definition fib(0) = 0
    if (n == 1) return 1; // BASE CASE: by definition fib(1) = 1

    // Two recursive calls — both shrink toward base case
    // n-1 and n-2 ensure we always move toward 0 or 1
    return fibRecursive(n - 1) + fibRecursive(n - 2);
}
// Time:  $O(2^n)$  — VERY SLOW — avoid for large n
// Space:  $O(n)$  — stack depth equals n
```

Why recursive fib is so slow — duplicated calls for fib(5):

```
fib(5)
├─ fib(4)
│   ├── fib(3) ← computed here
│   └─ fib(2)
└─ fib(3) ← computed AGAIN! Wasteful.
    ├── fib(2)
    └─ fib(1)
```

This doubling at every level =  $O(2^n)$  — use iterative instead!

### 6.2 Iterative Fibonacci — with variable explanations

```
// n = the position in the sequence we want
// prev2 = holds fib(i-2) — the value TWO steps back
//         starts as fib(0) = 0
// prev1 = holds fib(i-1) — the value ONE step back
//         starts as fib(1) = 1
// current = the value we are CURRENTLY computing = prev1 + prev2
//           this becomes the answer after the loop finishes
// i = loop counter, starts at 2 because we already know fib(0) and fib(1)
// i <= n = keep going until we reach the position we want
```

```

public static int fibIterative(int n) {
    if (n == 0) return 0; // handle base cases directly – no loop needed
    if (n == 1) return 1;

    int prev2 = 0;    // represents fib(0) = 0
    int prev1 = 1;    // represents fib(1) = 1
    int current = 0;  // will hold the new value each iteration

    for (int i = 2; i <= n; i++) {
        current = prev1 + prev2; // fib(i) = fib(i-1) + fib(i-2)
        prev2 = prev1;          // slide the window: old fib(i-1) becomes new
        // fib(i-2)
        prev1 = current;         // slide the window: new fib(i) becomes next
        // fib(i-1)
    }
    return current; // after loop, current holds fib(n)
}
// Time: O(n)    Space: O(1) – only 3 integer variables, never grows

```

| n | fib(n) | current = prev1 +<br>prev2 | prev2 → prev1    |
|---|--------|----------------------------|------------------|
| 0 | 0      | base case                  | —                |
| 1 | 1      | base case                  | —                |
| 2 | 1      | 1 + 0 = 1                  | prev2=1, prev1=1 |
| 3 | 2      | 1 + 1 = 2                  | prev2=1, prev1=2 |
| 4 | 3      | 2 + 1 = 3                  | prev2=2, prev1=3 |
| 5 | 5      | 3 + 2 = 5                  | prev2=3, prev1=5 |
| 6 | 8      | 5 + 3 = 8                  | prev2=5, prev1=8 |

## 7. Searching Algorithms

### 7.1 Linear Search — with variable explanations

Start at position 0. Check each element one by one until we find the target or run out of elements.

```
// arr    = the array we are searching through
// target = the value we are LOOKING FOR
// i      = current index/position we are checking
//         starts at 0 because arrays begin at index 0
// i < arr.length = stop when we have checked every slot
//         arr.length gives the total number of elements
// arr[i] = the element at position i — we compare this to target
// return i = we found target at position i, report back WHERE it is
// return -1 = -1 is a signal value meaning 'not found'
//         we use -1 because it is not a valid array index

public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++) { // check every slot, left to right
        if (arr[i] == target) {
            return i; // FOUND at index i — stop searching
        }
    }
    return -1; // Reached end without finding target
}

// EXAMPLE: arr={10,25,8,42,15}, target=42
// i=0: arr[0]=10 ≠ 42 | i=1: arr[1]=25 ≠ 42
// i=2: arr[2]=8  ≠ 42 | i=3: arr[3]=42 == 42 → return 3
```

| Case         | Complexity | Scenario                                                |
|--------------|------------|---------------------------------------------------------|
| Best Case    | O(1)       | Target is the FIRST element — return immediately        |
| Average Case | O(n)       | Target is somewhere in the middle                       |
| Worst Case   | O(n)       | Target is LAST or not found — must check ALL n elements |

### 7.2 Binary Search — with variable explanations

REQUIRES a sorted array. Each step cuts the search space in HALF by comparing target to the middle element.

```
// arr    = the SORTED array to search (must be sorted or this breaks!)
// target = value we are looking for
// low    = index of the LEFT boundary of our current search space
//         starts at 0 (beginning of array)
// high   = index of the RIGHT boundary of our current search space
//         starts at arr.length-1 (last element)
```

```

// mid    = the MIDDLE index of the current search space
//          calculated as (low + high) / 2
//          WHY? we compare arr[mid] to target to decide which HALF to keep
// low <= high = as long as there is still a range to search
//            when low > high, the range has collapsed - item not found

public static int binarySearch(int[] arr, int target) {
    int low = 0;           // start: left boundary = first index
    int high = arr.length - 1; // start: right boundary = last index

    while (low <= high) {    // keep going while search space exists
        int mid = (low + high) / 2; // middle of current range

        if (arr[mid] == target) {
            return mid;           // FOUND exactly at the middle

        } else if (arr[mid] < target) {
            low = mid + 1;         // target is BIGGER → discard left half
            // mid+1 because we already checked arr[mid], no need to include it

        } else {
            high = mid - 1;        // target is SMALLER → discard right half
            // mid-1 because we already checked arr[mid]
        }
    }
    return -1; // low > high: search space empty → not found
}

// EXAMPLE: arr={3,7,12,18,25,34,45}, target=34
// Step 1: low=0,high=6 → mid=3 → arr[3]=18 < 34 → low = 4
// Step 2: low=4,high=6 → mid=5 → arr[5]=34 == 34 → return 5 ✓

```

| Case         | Complexity  | Scenario                                  |
|--------------|-------------|-------------------------------------------|
| Best Case    | $O(1)$      | Target is the MIDDLE element on first try |
| Average Case | $O(\log n)$ | Each step halves the problem size         |
| Worst Case   | $O(\log n)$ | Target at the far end or not found        |

## 7.3 Linear vs Binary Search

|                    | Linear Search     | Binary Search         |
|--------------------|-------------------|-----------------------|
| Needs sorted data? | NO                | YES — must be sorted  |
| Best Case          | $O(1)$            | $O(1)$                |
| Worst Case         | $O(n)$            | $O(\log n)$           |
| Space              | $O(1)$            | $O(1)$                |
| When to use?       | Small or unsorted | Large sorted datasets |



## 8. Bubble Sort

Compare NEIGHBOURING elements. If left > right, swap them. After each full pass, the largest unsorted element 'bubbles up' to its correct position at the right end.

### 8.1 Java Code — with variable explanations

```
// arr      = the array we want to sort (it is modified IN PLACE - no copy made)
// n        = total number of elements in the array (arr.length)
// i        = outer loop counter - tracks which PASS we are on
//           Pass 0 puts the largest at the end
//           Pass 1 puts the second largest at arr[n-2], etc.
//           i < n-1 because after n-1 passes the whole array is sorted
// swapped   = a boolean FLAG - starts false each pass
//           set to true if ANY swap happens during this pass
//           if swapped is still false after a full pass, array is already sorted
//           this 'early exit' turns best case from O(n²) to O(n)
// j        = inner loop counter - the LEFT element of the pair we are comparing
//           j starts at 0 (first pair: arr[0] and arr[1])
// j < n-1-i = WHY n-1-i? After pass i, the last i elements are already sorted
//           so we don't need to compare them again - saves time
//           e.g. after pass 0: last 1 element sorted → stop at n-2
//           after pass 1: last 2 elements sorted → stop at n-3, etc.
// arr[j]    = the LEFT element of the current pair
// arr[j+1]  = the RIGHT element of the current pair
//           j+1 is j's neighbour - one slot to the right
// temp      = a temporary holder so we don't lose arr[j]'s value during swap
//           swap always needs 3 steps: save A → overwrite A with B → put saved A
//           into B

public static void bubbleSort(int[] arr) {
    int n = arr.length;           // total element count

    for (int i = 0; i < n - 1; i++) { // n-1 passes needed at most
        boolean swapped = false;    // assume sorted until a swap happens

        for (int j = 0; j < n - 1 - i; j++) { // compare unsorted pairs
            if (arr[j] > arr[j + 1]) { // left neighbour bigger than right?

                // SWAP arr[j] and arr[j+1]
                int temp = arr[j];    // step 1: save arr[j] before overwriting
                arr[j] = arr[j + 1];  // step 2: put right element into left
                slot
                arr[j + 1] = temp;    // step 3: put saved left element into
                right slot

                swapped = true;        // record that a swap happened this pass
            }
        }

        if (!swapped) break; // no swaps = already sorted → exit early (Best Case
        O(n))
    }
}
```

## 8.2 Step-by-Step Trace — {90, 23, 5, 109, 12}

| Step        | Pair compared           | Action                         | Array state after    |
|-------------|-------------------------|--------------------------------|----------------------|
| Pass 1, j=0 | arr[0]=90 vs arr[1]=23  | 90 > 23 → SWAP                 | [23, 90, 5, 109, 12] |
| Pass 1, j=1 | arr[1]=90 vs arr[2]=5   | 90 > 5 → SWAP                  | [23, 5, 90, 109, 12] |
| Pass 1, j=2 | arr[2]=90 vs arr[3]=109 | 90 < 109 → No swap             | [23, 5, 90, 109, 12] |
| Pass 1, j=3 | arr[3]=109 vs arr[4]=12 | 109 > 12 → SWAP —<br>109 done! | [23, 5, 90, 12, 109] |
| Pass 2, j=0 | arr[0]=23 vs arr[1]=5   | 23 > 5 → SWAP                  | [5, 23, 90, 12, 109] |
| Pass 2, j=1 | arr[1]=23 vs arr[2]=90  | 23 < 90 → No swap              | [5, 23, 90, 12, 109] |
| Pass 2, j=2 | arr[2]=90 vs arr[3]=12  | 90 > 12 → SWAP —<br>90 done!   | [5, 23, 12, 90, 109] |
| Pass 3, j=0 | arr[0]=5 vs arr[1]=23   | 5 < 23 → No swap               | [5, 23, 12, 90, 109] |
| Pass 3, j=1 | arr[1]=23 vs arr[2]=12  | 23 > 12 → SWAP —<br>23 done!   | [5, 12, 23, 90, 109] |
| Pass 4, j=0 | arr[0]=5 vs arr[1]=12   | 5 < 12 → No swap →<br>DONE ✓   | [5, 12, 23, 90, 109] |

## 8.3 Complexity Analysis

| Case         | Complexity | Why                                               |
|--------------|------------|---------------------------------------------------|
| Best Case    | $O(n)$     | Already sorted — 1 pass, swapped flag exits early |
| Average Case | $O(n^2)$   | Random order — $\sim n^2/2$ comparisons           |
| Worst Case   | $O(n^2)$   | Reverse sorted — $n(n-1)/2$ comparisons max       |
| Space        | $O(1)$     | In-place — only uses one temp variable            |

## 9. Insertion Sort

Pick the next unsorted element and SLIDE IT LEFT through the already-sorted portion until it lands in the right spot. Like sorting playing cards in your hand.

### 9.1 Java Code — with variable explanations

```
// arr      = the array we are sorting (modified in place)
// i        = outer loop counter — the INDEX of the element we are about to insert
//           WHY start at 1? Because a single element (arr[0]) is trivially
'sorted'
//           We grow the sorted region one element at a time from index 1 onward
// key      = the element we picked up and are about to insert into the sorted
region
//           WHY store it separately? Because we are going to OVERWRITE arr[i]
//           during the shifting process, so we must save the original value first
//           Think of 'key' as the card you picked up from the unsorted deck
// j        = index of the last element in the sorted region
//           starts at i-1 because that is the element just LEFT of our key
//           j scans BACKWARD (decreasing) through the sorted region
// j >= 0   = don't go past the beginning of the array (index -1 does not exist)
// arr[j] > key = is the element at j BIGGER than our key?
//           if yes, it needs to slide one place to the RIGHT to make room
//           if no, we have found where key belongs — stop sliding
// arr[j+1] = arr[j] means: copy the element at j one position to the right
//           j+1 is the slot WE came from — it is currently empty/overwritable
//           this is the 'shifting right' action
// j--      = move j one step LEFT to check the next element in the sorted region
// arr[j+1] = key means: place key into the slot we stopped at
//           after the while loop, j+1 is the correct sorted position for key
//           WHY j+1 not j? Because the final j-- overshoot by one

public static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) { // i=1: start with second element
        int key = arr[i]; // save the element we want to insert
        int j = i - 1;    // j starts at the element just before key

        // Shift all sorted elements that are BIGGER than key one slot right
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j]; // slide arr[j] one position right
            j--;                // move one step left to check next element
        }
        // j is now at the last element that is <= key (or j = -1 if key is
        // smallest)
        // so j+1 is the correct insertion point
        arr[j + 1] = key; // place key in its sorted position
    }
}
```

### 9.2 Step-by-Step Trace — {90, 23, 5, 109, 12}

| i (pass) | key saved | What happens | Array after |
|----------|-----------|--------------|-------------|
|----------|-----------|--------------|-------------|

|     |         |                                                                                                            |                        |
|-----|---------|------------------------------------------------------------------------------------------------------------|------------------------|
| i=1 | key=23  | j=0: arr[0]=90 > 23 →<br>slide 90 right. j=-1 →<br>insert 23 at j+1=0                                      | [23, 90, 5, 109, 12]   |
| i=2 | key=5   | j=1: 90>5 → slide. j=0:<br>23>5 → slide. j=-1 →<br>insert 5 at 0                                           | [5, 23, 90, 109, 12]   |
| i=3 | key=109 | j=2: arr[2]=90 < 109 →<br>stop. Insert at j+1=3<br>(stays in place)                                        | [5, 23, 90, 109, 12]   |
| i=4 | key=12  | j=3: 109>12 → slide.<br>j=2: 90>12 → slide.<br>j=1: 23>12 → slide.<br>j=0: 5<12 → stop.<br>Insert at j+1=1 | [5, 12, 23, 90, 109] ✓ |

## 9.3 Complexity Analysis

| Case         | Complexity | Why                                                              |
|--------------|------------|------------------------------------------------------------------|
| Best Case    | $O(n)$     | Already sorted — inner while never executes, just i loop         |
| Average Case | $O(n^2)$   | Random data — $\sim n/2$ shifts per element on average           |
| Worst Case   | $O(n^2)$   | Reverse sorted — max shifts (j goes all the way to 0) every time |
| Space        | $O(1)$     | In-place — only 'key' and 'j' as extra variables                 |

## 10. Quick Sort

---

DIVIDE and CONQUER. Pick a PIVOT element, rearrange the array so everything smaller goes left of pivot and everything larger goes right, then recursively sort both halves.

### 10.1 Steps

1. Choose a PIVOT (last element, first, or middle — we use last here)
2. PARTITION: rearrange so all < pivot are left, all > pivot are right
3. RECURSIVELY apply to left sub-array and right sub-array
4. BASE CASE: sub-array of size 0 or 1 is already sorted — do nothing

### 10.2 Java Code — with variable explanations

```
// arr = the array being sorted (modified in place)
// low = the starting index of the sub-array we are currently sorting
//       first call: low = 0 (beginning of array)
//       recursive calls: low is the start of a smaller sub-section
// high = the ending index of the sub-array we are currently sorting
//       first call: high = arr.length - 1 (end of array)
// low < high = base case check: if low == high, only one element → already sorted
//       if low > high, empty range → nothing to do
// pivotIndex = the position where pivot LANDED after partitioning
//       everything left of pivotIndex is <= pivot
//       everything right of pivotIndex is >= pivot
//       pivot itself is now in its FINAL sorted position

public static void quickSort(int[] arr, int low, int high) {
    if (low < high) { // only sort if range has more than 1 element
        int pivotIndex = partition(arr, low, high);
        quickSort(arr, low, pivotIndex - 1); // recursively sort LEFT half
        quickSort(arr, pivotIndex + 1, high); // recursively sort RIGHT half
        // pivotIndex itself is excluded - it is already in its correct place
    }
}

// ——— partition() ———
// pivot = the element chosen as pivot (we always pick arr[high] - last element)
// goal: move everything < pivot to left, everything > pivot to right
// i = the 'boundary' index - everything at or left of i is <= pivot
//     starts at low-1 meaning 'nothing in the small-elements zone yet'
// j = scans from low to high-1 (we skip high because that IS the pivot)
//     j checks every element and decides: does it belong left of pivot?
// i++ = expand the small-elements zone by one slot
// swap arr[i] and arr[j] = move the element that belongs left of pivot into place
// final swap = move pivot from arr[high] to arr[i+1] which is its correct position
// return i+1 = the pivot's final index (used to split the next recursive calls)

public static int partition(int[] arr, int low, int high) {
    int pivot = arr[high]; // choose last element as pivot
    int i = low - 1;       // boundary: starts before the sub-array

    for (int j = low; j < high; j++) { // j scans every element except pivot
```

```

    if (arr[j] <= pivot) {           // this element belongs LEFT of pivot
        i++;                        // expand the left zone by one
        // swap arr[i] and arr[j] to move small element into left zone
        int temp = arr[i]; arr[i] = arr[j]; arr[j] = temp;
    }
    // if arr[j] > pivot: do nothing - j just moves on
}
// now place pivot between the two zones
int temp = arr[i + 1]; arr[i + 1] = arr[high]; arr[high] = temp;
return i + 1; // pivot's final sorted index
}

// To sort the whole array: quickSort(arr, 0, arr.length - 1);

```

### 10.3 Trace — {90, 23, 5, 109, 12}

| Step | Pivot       | Action                             | Result               |
|------|-------------|------------------------------------|----------------------|
| R1   | 12 (arr[4]) | < 12: {5}   12   > 12: {90,23,109} | [5   12   90,23,109] |
| R2   | —           | {5}: size 1 → sorted (base case)   | [5, 12, ...]         |
| R3   | 109 (last)  | < 109: {90,23}   109   > 109: {}   | [..., 90,23   109]   |
| R4   | 23 (last)   | < 23: {}   23   > 23: {90}         | [..., 23, 90, 109]   |
| DONE | —           | All sub-arrays size ≤ 1 → SORTED ✓ | [5, 12, 23, 90, 109] |

### 10.4 Complexity Analysis

| Case         | Complexity    | Why                                                                |
|--------------|---------------|--------------------------------------------------------------------|
| Best Case    | $O(n \log n)$ | Pivot always splits into equal halves → $\log n$ levels × $n$ work |
| Average Case | $O(n \log n)$ | Roughly balanced splits on random data                             |
| Worst Case   | $O(n^2)$      | Sorted array + last-element pivot → one side always empty          |
| Space        | $O(\log n)$   | Recursive call stack depth ( $\log n$ levels)                      |

**TIP:** To avoid worst case, use RANDOM PIVOT or MEDIAN-OF-THREE pivot.

# 11. Heap Sort

Uses a MAX-HEAP — a binary tree structure where every PARENT is GREATER than or equal to its children. The ROOT is always the MAXIMUM element.

Important: We represent the heap inside the SAME array — no extra data structure needed.

Array-to-Tree mapping (zero-indexed):

Parent of node  $i$  =  $(i - 1) / 2$

Left child of node  $i$  =  $2*i + 1$

Right child of node  $i$  =  $2*i + 2$

Last non-leaf node =  $(n/2) - 1$  (start heapifying from here)

## 11.1 Java Code — with variable explanations

```
// arr    = array to sort (sorted in place)
// n      = total number of elements

public static void heapSort(int[] arr) {
    int n = arr.length;

    // — PHASE 1: Build Max-Heap —————
    // We start from the LAST non-leaf node and heapify upward
    // WHY (n/2)-1? Nodes at index n/2 to n-1 are all LEAF nodes
    //           leaves have no children so they are already valid heaps
    //           we only need to heapify the nodes that HAVE children
    // i-- = process each internal node, going RIGHT to LEFT (bottom up)
    for (int i = n / 2 - 1; i >= 0; i--) {
        heapify(arr, n, i); // fix the heap rooted at index i
    }

    // — PHASE 2: Extract elements one by one —————
    // i    = index of the LAST element in the 'unsorted heap' portion
    //       starts at n-1 (last element) and shrinks after each extraction
    // i-- = reduce the heap size: the extracted max is now 'sorted' at the end
    for (int i = n - 1; i > 0; i--) {
        // Swap: move current root (max) to the END of unsorted portion
        // arr[0] = root = maximum value of current heap
        // arr[i] = last unsorted element - it takes root's place temporarily
        int temp = arr[0]; arr[0] = arr[i]; arr[i] = temp;

        // Re-heapify the reduced heap (size i, root at 0)
        // The new root (formerly arr[i]) is probably not the max - fix it
        heapify(arr, i, 0);
    }
}

// — heapify() —————
// arr    = the array (used as a tree)
// n      = the current heap SIZE (shrinks in Phase 2)
// i      = the ROOT index of the subtree we want to fix
// largest = index of the largest among: node i, left child, right child
//           starts as i (assume i is largest), then check children
// left   = index of i's left child = 2*i + 1
// right  = index of i's right child = 2*i + 2
```

```
// WHY check left < n and right < n?
//   Because a child index might be beyond the current heap size
//   (when the heap has shrunk in Phase 2) – must not access out-of-bounds

public static void heapify(int[] arr, int n, int i) {
    int largest = i;    // start by assuming parent is largest
    int left    = 2*i+1; // left child index
    int right   = 2*i+2; // right child index

    // Is left child within bounds AND bigger than current largest?
    if (left < n && arr[left] > arr[largest])
        largest = left;

    // Is right child within bounds AND bigger than current largest?
    if (right < n && arr[right] > arr[largest])
        largest = right;

    // If largest is NOT the parent, swap and continue fixing downward
    if (largest != i) {
        int temp = arr[i]; arr[i] = arr[largest]; arr[largest] = temp;
        heapify(arr, n, largest); // recursively fix the subtree that was disturbed
    }
}
```

## 11.2 Trace — {90, 23, 5, 109, 12}

### Array as Binary Tree:

```

    90 (index 0)
   / \
  23  5 (index 1, 2)
 / \
109 12 (index 3, 4)
```

### Phase 1 — Build Max-Heap (n=5, start from index 1):

| Step | Node (index)     | Action                                                      | Array after                        |
|------|------------------|-------------------------------------------------------------|------------------------------------|
| 1    | index 1 (val=23) | Children: index3=109, index4=12. 109 > 23 → SWAP 23 and 109 | [90, 109, 5, 23, 12]               |
| 2    | index 0 (val=90) | Children: index1=109, index2=5. 109 > 90 → SWAP 90 and 109  | [109, 90, 5, 23, 12]<br>MAX-HEAP ✓ |

### Phase 2 — Extract max n-1 times:

| Step | Action                                            | Array (  = boundary) | Heap portion           |
|------|---------------------------------------------------|----------------------|------------------------|
| E1   | Swap root 109 with arr[4]=12. Heapify 4 elements. | [12,90,5,23   109]   | Heapify → [90,23,5,12] |

|      |                                                  |                      |                     |
|------|--------------------------------------------------|----------------------|---------------------|
| E2   | Swap root 90 with arr[3]=12. Heapify 3 elements. | [12,23,5   90,109]   | Heapify → [23,12,5] |
| E3   | Swap root 23 with arr[2]=5. Heapify 2 elements.  | [5,12   23,90,109]   | Heapify → [12,5]    |
| E4   | Swap root 12 with arr[1]=5. Done.                | [5   12,23,90,109]   | Only 1 left         |
| DONE | Array fully sorted ✓                             | [5, 12, 23, 90, 109] | —                   |

## 11.3 Complexity Analysis

| Case         | Complexity    | Why                                                                 |
|--------------|---------------|---------------------------------------------------------------------|
| Best Case    | $O(n \log n)$ | Always the same — tree structure guarantees $O(\log n)$ per heapify |
| Average Case | $O(n \log n)$ | Consistent for any input                                            |
| Worst Case   | $O(n \log n)$ | GUARANTEED — never degrades to $O(n^2)$ unlike QuickSort            |
| Build Heap   | $O(n)$        | Phase 1 is linear (amortised analysis)                              |
| Space        | $O(1)$        | In-place — no extra array needed                                    |

## 12. Sorting Algorithms — Complete Comparison

| Algorithm      | Best Case     | Average Case  | Worst Case    | Space       | Stable? |
|----------------|---------------|---------------|---------------|-------------|---------|
| Bubble Sort    | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes     |
| Insertion Sort | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$      | Yes     |
| Quick Sort     | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$ | No      |
| Heap Sort      | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$      | No      |

STABLE sort: equal elements keep their original relative order.

IN-PLACE sort: uses  $O(1)$  extra space (only a few swap variables).

Heap Sort is the **ONLY** one here that is **ALWAYS**  $O(n \log n)$  — no worst case.

Quick Sort is usually **FASTEST** in practice despite an  $O(n^2)$  worst case.

Bubble & Insertion are only good for **SMALL** or **NEARLY-SORTED** datasets.

## 13. Linked Lists

A Linked List is a DYNAMIC data structure made of NODES. Each node holds DATA and a POINTER to the next node. Nodes can be anywhere in memory — they do not need contiguous slots.

### 13.1 Array vs Linked List

| Operation       | Array              | Linked List                          |
|-----------------|--------------------|--------------------------------------|
| Access by index | $O(1)$ — direct    | $O(n)$ — must walk from head         |
| Insert at front | $O(n)$ — shift all | $O(1)$ — update head pointer only    |
| Insert at end   | $O(1)$             | $O(n)$ unless tail pointer is stored |
| Delete middle   | $O(n)$ — shift     | $O(n)$ to find + $O(1)$ to remove    |
| Size            | Fixed              | Dynamic — grows/shrinks freely       |

### 13.2 Singly Linked List — with variable explanations

HEAD → [Data|next] → [Data|next] → [Data|next] → null  
Node 1      Node 2      Node 3

Example: HEAD → [10|•] → [20|•] → [30|null]

'head' always points to the first node.

The last node's 'next' is null — that signals the end of the list.

```
// — Node class —————
// data = the actual VALUE stored in this node (e.g. 10, 20, 30)
// next = a REFERENCE (pointer) to the NEXT node in the list
//       if this is the last node, next = null (no node after it)

class Node {
    int data; // the value stored here
    Node next; // pointer to the next node (null if last)

    public Node(int data) {
        this.data = data;
        this.next = null; // new node has no successor yet
    }
}

// — LinkedList class —————
// head = pointer to the FIRST node in the list
//       if head == null, the list is empty

class LinkedList {
    Node head; // always points to the start of the list
}
```

```

// INSERT AT FRONT - O(1)
// newNode = the new Node we are adding
// newNode.next = head means: the new node points to the OLD first node
// head = newNode means: head now considers newNode the first node
// Result: newNode is at the front, old head is now second
public void insertAtFront(int data) {
    Node newNode = new Node(data);
    newNode.next = head; // new node links to old head
    head = newNode;     // update head to point at new node
}

// INSERT AT END - O(n)
// current = a 'traveller' pointer that walks the list
//           starts at head and follows .next until .next == null
// current.next == null means we are at the LAST node
// current.next = newNode links the last node to the new one
public void insertAtEnd(int data) {
    Node newNode = new Node(data);
    if (head == null) { head = newNode; return; } // empty list
    Node current = head;
    while (current.next != null) { // walk to last node
        current = current.next;
    }
    current.next = newNode; // attach new node at end
}

// DISPLAY - O(n)
// current = traveller starting at head
// current != null = keep going while there is still a node
// current = current.next = move to the next node
public void display() {
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next; // advance to next node
    }
    System.out.println("null"); // signal end of list
}

// DELETE by value - O(n)
// current = traveller to find the node BEFORE the one we want to delete
// current.next.data == data = the NEXT node is the one to delete
// current.next = current.next.next = SKIP over the deleted node
//           the deleted node becomes unreachable -> garbage collected
public void delete(int data) {
    if (head == null) return; // empty list
    if (head.data == data) { head = head.next; return; } // delete head
    Node current = head;
    while (current.next != null && current.next.data != data) {
        current = current.next;
    }
    if (current.next != null) {
        current.next = current.next.next; // bypass deleted node
    }
}
}

```

## 13.3 Doubly Linked List — with variable explanations

Each node has a PREV pointer going backward AND a NEXT pointer going forward. Traverse in BOTH directions.

```
null <- [prev|Data|next] <-> [prev|Data|next] <-> [prev|Data|next] -> null
      Node 1           Node 2           Node 3
```

Each node knows its PREVIOUS neighbour AND its NEXT neighbour.

```
// — Doubly Node —
// prev = pointer to the PREVIOUS node (null for first node)
// data = the stored value
// next = pointer to the NEXT node (null for last node)

class DNode {
    int data;
    DNode prev; // points BACKWARD to previous node
    DNode next; // points FORWARD to next node

    public DNode(int data) {
        this.data = data;
        this.prev = null; // no predecessor yet
        this.next = null; // no successor yet
    }
}

class DoublyLinkedList {
    DNode head; // first node
    DNode tail; // last node — kept for O(1) end insertion

    // INSERT AT END — O(1) because we store tail
    // newNode.prev = tail → new node looks BACK at old tail
    // tail.next = newNode → old tail looks FORWARD to new node
    // tail = newNode → update tail reference to new last node
    public void insertAtEnd(int data) {
        DNode newNode = new DNode(data);
        if (head == null) { head = tail = newNode; return; }
        newNode.prev = tail; // link backward: new → old tail
        tail.next = newNode; // link forward: old tail → new
        tail = newNode; // advance tail pointer
    }

    // TRAVERSE BACKWARD (unique to doubly lists!)
    // current starts at tail (last node) and follows .prev
    // current != null = stop when we pass the first node
    public void displayReverse() {
        DNode current = tail; // start at the END
        while (current != null) {
            System.out.print(current.data + " ");
            current = current.prev; // go BACKWARDS
        }
    }
}
```

## 13.4 Circular Linked List — with variable explanations

The LAST node's next pointer circles back to HEAD instead of null. There is no true end — it forms a ring.

HEAD -> [10|next] -> [20|next] -> [30|next]

^  
(last node's next points back to HEAD — creates the circle)

Useful for: round-robin scheduling, game turns, circular queues, music playlists.

```
class CircularLinkedList {
    Node head; // points to first node

    // INSERT AT END - O(n) to find last node
    // Special case: if head == null, the list is empty
    // → the new node's next must point to ITSELF (only node = circular already)
    // current.next != head = stop when we reach the node whose next IS head
    // that node IS the last node currently
    // current.next = newNode → last node now points to new node
    // newNode.next = head → new node circles back to head
    public void insertAtEnd(int data) {
        Node newNode = new Node(data);
        if (head == null) {
            head = newNode;
            head.next = head; // only node — points to itself
            return;
        }
        Node current = head;
        while (current.next != head) { // walk to last node
            current = current.next;
        }
        current.next = newNode; // last node → new node
        newNode.next = head;    // new node → head (closes the circle!)
    }

    // DISPLAY — uses do-while so we visit head at least once
    // WHY do-while and not while?
    // A regular while checks 'current != head' BEFORE printing
    // But current STARTS as head → regular while would exit immediately!
    // do-while: print FIRST, then check → guaranteed to visit every node
    public void display() {
        if (head == null) return;
        Node current = head;
        do {
            System.out.print(current.data + " -> ");
            current = current.next; // advance
        } while (current != head); // stop when we loop back to head
        System.out.println("(back to HEAD)");
    }
}
```

## 13.5 Summary of Linked List Types

| Type          | Traverse             | Last node.next | Key advantage                      |
|---------------|----------------------|----------------|------------------------------------|
| Singly Linked | Forward only         | null           | Simple, low memory per node        |
| Doubly Linked | Forward AND backward | null           | Delete/traverse in both directions |

|                 |                        |           |                                         |
|-----------------|------------------------|-----------|-----------------------------------------|
| Circular Singly | Forward (wraps)        | HEAD      | Round-robin, no end to worry about      |
| Circular Doubly | Both directions (wrap) | HEAD/TAIL | Full flexibility, used in OS schedulers |

## 14. Master Quick-Reference — Complexity Cheat Sheet

| Algorithm / Structure | Best          | Average       | Worst         | Space              | Key Note                 |
|-----------------------|---------------|---------------|---------------|--------------------|--------------------------|
| Linear Search         | $O(1)$        | $O(n)$        | $O(n)$        | $O(1)$             | Works on unsorted data   |
| Binary Search         | $O(1)$        | $O(\log n)$   | $O(\log n)$   | $O(1)$             | Needs SORTED array       |
| Bubble Sort           | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$             | Stable, in-place         |
| Insertion Sort        | $O(n)$        | $O(n^2)$      | $O(n^2)$      | $O(1)$             | Stable, in-place         |
| Quick Sort            | $O(n \log n)$ | $O(n \log n)$ | $O(n^2)$      | $O(\log n)$        | Fastest in practice      |
| Heap Sort             | $O(n \log n)$ | $O(n \log n)$ | $O(n \log n)$ | $O(1)$             | Guaranteed $O(n \log n)$ |
| Factorial (Recur)     | $O(n)$        | $O(n)$        | $O(n)$        | $O(n)$             | Risk: StackOverflow      |
| Factorial (Iter)      | $O(n)$        | $O(n)$        | $O(n)$        | $O(1)$             | Always preferred         |
| Fib (Recursive)       | $O(2^n)$      | $O(2^n)$      | $O(2^n)$      | $O(n)$             | VERY slow! Avoid         |
| Fib (Iterative)       | $O(n)$        | $O(n)$        | $O(n)$        | $O(1)$             | Always preferred         |
| Linked List insert    | $O(1)$        | $O(n)$        | $O(n)$        | $O(1)/\text{node}$ | Front insert = $O(1)$    |
| Linked List search    | $O(1)$        | $O(n)$        | $O(n)$        | $O(1)$             | No index access          |

COMPLEXITY ORDER (fastest → slowest):

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(2^n)$

BIG O RULES — memorise these:

- Sequential loops → ADD costs, then drop constants:  $O(n) + O(n) = O(n)$
- Nested loops → MULTIPLY costs:  $O(n) \times O(n) = O(n^2)$
- Halving/tripling → LOGARITHMIC inner loop:  $O(\log n)$
- Recursion → Each call = one stack frame:  $O(n)$  space
- Drop constants →  $O(3n^2) = O(n^2)$  |  $O(n/4) = O(n)$

VARIABLE CHEAT SHEET for sorting code:

n = array length    arr = the array being sorted  
i = outer pass/position    j = inner comparison index  
key = element being inserted (Insertion Sort)  
temp = swap helper (always needed for 3-step swap)  
pivot = chosen element in Quick Sort  
low/high = boundaries of current sub-array (Quick Sort)

Good luck on Thursday, Thabiso! You've got this! 🍀